Chapter 3

implement various data structures and their algorithms, and apply them in implementing

Building upon the abstract data types and object-oriented design patterns introduced in prior chapters, software systems require concrete underlying implementations to manage memory and execute operations efficiently. While high-level programming environments provide built-in data structures, constructing these structures directly exposes fundamental trade-offs between memory layout, access overhead, and algorithmic complexity.

This chapter examines the internal mechanics, structural implementations, and algorithmic operations of core data structures: linked lists, stacks, queues, binary search trees, and hash tables. Mastering these low-level implementations provides the operational insight needed to evaluate execution bottlenecks, avoid hidden overheads, and build custom data organizations for complex software applications.

3.1 Pointer-Based Linear Structures: Linked Lists

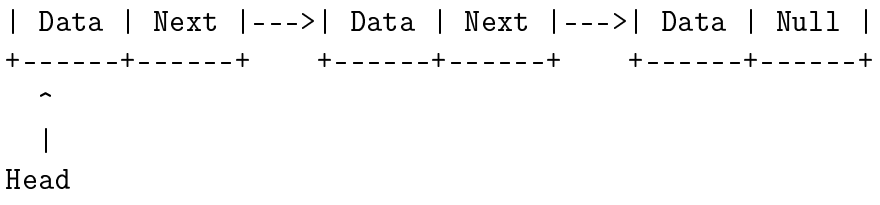
Linear data structures organize elements in a sequential order. Arrays allocate contiguous blocks of memory, whereas pointer-based lists distribute elements dynamically across non-contiguous physical memory locations.

3.1.1 Singly Linked Lists

A **singly linked list** consists of a sequence of nodes where each node contains two distinct fields: a data element and a reference pointer to the next node in the sequence. The structure is anchored by a `head` pointer that points to the first node. The terminal node contains a `null` reference to signal the end of the list.

```
Singly Linked List Memory Layout:

 +------+------+    +------+------+    +------+------+
```

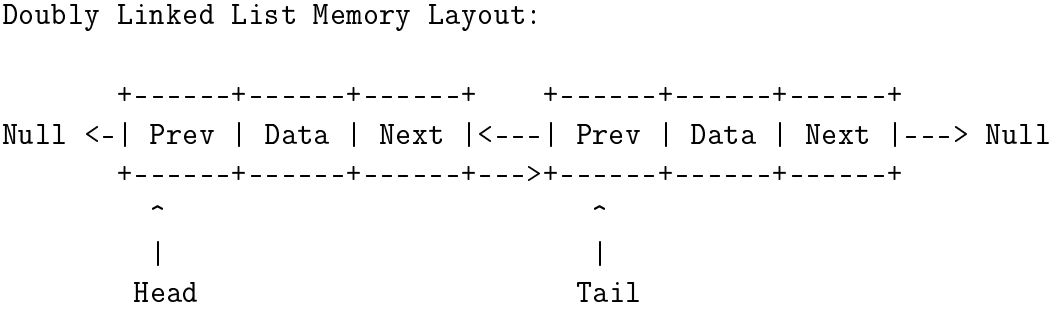


Primary operations on a singly linked list operate via pointer manipulation:

- **Insertion at Head**: A new node is allocated, its `next` pointer is directed to the current `head`, and the `head` pointer is updated to point to the new node. This operates in $O(1)$ constant time.
- **Traversal and Search**: Accessing an element at index k requires traversing k pointer references sequentially starting from `head`. This operation requires $O(n)$ linear time.
- **Deletion**: Removing a target node requires updating the `next` pointer of its preceding node to bypass the target node and point directly to the target node's successor. Locating the predecessor requires $O(n)$ search time, but the link modification itself is $O(1)$.

3.1.2 Doubly Linked Lists

A **doubly linked list** expands the node design by incorporating two pointers per node: a `next` pointer referencing the successor node and a `prev` pointer referencing the predecessor node. A `tail` pointer is maintained alongside the `head` pointer to anchor both ends of the structure.



Bidirectional pointers eliminate the need to traverse from the head when searching for a preceding node during deletion. Given a direct pointer reference to a node P within a doubly linked list, P can be removed in $O(1)$ time by setting $P.\texttt{prev}.\texttt{next} = P.\texttt{next}$ and $P.\texttt{next}.\texttt{prev} = P.\texttt{prev}$. However, this additional pointer maintenance increases the per-node dynamic memory overhead.

3.1.3 Worked Example: Deletion in a Doubly Linked List

Consider a doubly linked list containing nodes $A \leftrightarrow B \leftrightarrow C$. To delete target node B:

1. Access node A via $B.\texttt{prev}$ and node C via $B.\texttt{next}$.
2. Update node A's forward reference: $A.\texttt{next} \leftarrow C$.

3. Update node C's backward reference: C.`prev` $\leftarrow A$.
4. Clear node B's internal pointers: B.`next` $\leftarrow$ `null`, B.`prev` $\leftarrow$ `null`.
5. Release memory allocated for node B.

3.2 Restricted Linear Structures: Stacks and Queues

Stacks and queues apply specific constraints to element access and ordering, abstracting insertion and removal into well-defined interface behaviors.

3.2.1 Stacks

A **stack** is a linear data structure operating under a Last-In, First-Out (LIFO) protocol. Elements can only be added or removed from a single end, designated as the top of the stack.

The core operations are:

- `push(item)`: Places a new item onto the top of the stack.
- `pop()`: Removes and returns the top item from the stack.
- `peek()`: Returns the top item without modifying the stack.

A stack can be implemented using either a dynamic array or a singly linked list. In an array-based implementation, the top is tracked by an integer index, yielding $O(1)$ amortized push operations and $O(1)$ pop operations. In a linked list implementation, push and pop operations perform node insertion and removal at the list head in deterministic $O(1)$ time.

```
Stack Conceptual Structure:

      +----------+
      |  Item C  |  <-- Top of Stack (Push / Pop)
      +----------+
      |  Item B  |
      +----------+
      |  Item A  |
      +----------+
```

3.2.2 Queues

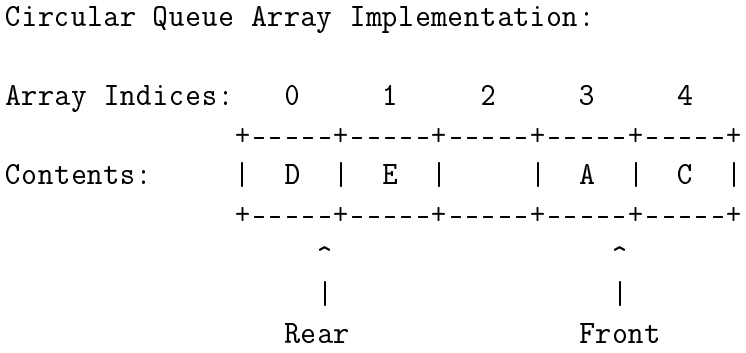
A **queue** is a linear data structure operating under a First-In, First-Out (FIFO) protocol. Elements are inserted at the rear and removed from the front.

The core operations are:

- `enqueue(item)`: Appends an element to the rear of the queue.
- `dequeue()`: Removes and returns the element at the front of the queue.

Implementing a queue via a standard contiguous array can cause performance issues if dequeue operations shift remaining elements left, incurring $O(n)$ cost per operation. To maintain $O(1)$ operations, arrays are implemented as **circular queues**, where two index pointers (`front` and `rear`) wrap around the physical boundaries using modular arithmetic:

$$\text{rear} = (\text{rear} + 1) \pmod{\text{capacity}}$$



3.2.3 Worked Example: Postfix Expression Evaluation using a Stack

Postfix notation (Reverse Polish Notation) eliminates the need for parentheses in arithmetic expressions by placing operators immediately after their operands. Evaluators process postfix strings sequentially from left to right using a stack:

- If a value is an operand, push it onto the stack.
- If a value is an operator, pop two operands from the stack, apply the operator (the second popped value is the left operand), and push the result back onto the stack.

Evaluate the postfix expression: $6 \quad 2 \quad 3 \quad * \quad + \quad 4 \quad -$

Table 3.1: Step-by-step Evaluation of $6 \quad 2 \quad 3 \quad * \quad + \quad 4 \quad -$

Step	**Token**	**Action**	**Stack State (Bottom to Top)**
1	6	Push 6	$[6]$
2	2	Push 2	$[6, 2]$
3	3	Push 3	$[6, 2, 3]$
4	$*$	Pop $3, 2$; Evaluate $2 * 3 = 6$; Push 6	$[6, 6]$
5	$+$	Pop $6, 6$; Evaluate $6 + 6 = 12$; Push 12	$[12]$
6	4	Push 4	$[12, 4]$
7	$-$	Pop $4, 12$; Evaluate $12 - 4 = 8$; Push 8	$[8]$

The evaluation completes with a single final value 8 remaining on top of the stack.

3.3 Nonlinear Structures: Binary Search Trees

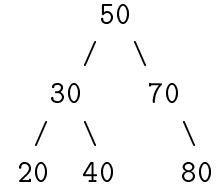
Linear data structures require linear search time $O(n)$ for unsorted data. Hierarchical structures optimize search, insertion, and deletion times through branched structural paths.

3.3.1 Binary Search Tree Property

A **binary search tree** (BST) is a hierarchical, node-based tree structure. Each node contains a key, a satellite payload, a left reference pointer, and a right reference pointer. A binary tree satisfies the binary search tree property if for every node N:

- Every node key in the left subtree of N is strictly less than N's key.
- Every node key in the right subtree of N is strictly greater than N's key.

```
Binary Search Tree Layout:
```



3.3.2 Core Operations and Algorithms

Search and Insertion

Searching begins at the root node. The search key is compared against the current node key:

- If equal, the node is returned.
- If the search key is smaller, search recursively continues into the left child subtree.
- If the search key is larger, search recursively continues into the right child subtree.

Insertion follows the same structural navigation path until reaching a `null` link, where the new node is attached as a leaf node.

Deletion

Tree node deletion must preserve the BST ordering invariant across three possible node configuration cases:

1. **Node has no children (Leaf Node)**: Remove the node directly by setting its parent reference to `null`.
2. **Node has one child**: Replace the target node with its single child by pointing the target node's parent directly to that child.

3. **Node has two children**: Locate the node's **in-order successor** (the node with the smallest key in its right subtree). Copy the in-order successor's key and satellite payload into the target node, then recursively delete the in-order successor node from its original position (which is guaranteed to fall under Case 1 or Case 2).

```
Deletion of Node with Two Children (Deleting Node 30):

       50                      50
      /  \                    /  \
    30    70   =====>       40    70
   /  \     \               /        \
  20  40    80            20         80
```

3.3.3 Tree Traversals

Tree traversal algorithms visit every node in a tree systematically in $O(n)$ time:

- **In-Order Traversal (Left, Node, Right)**: Recursively visits the left subtree, processes the current node, and visits the right subtree. On a BST, this outputs keys in ascending sorted order.
- **Pre-Order Traversal (Node, Left, Right)**: Processes the current node before visiting its subtrees. Used to clone or serialize tree topologies.
- **Post-Order Traversal (Left, Right, Node)**: Processes both subtrees prior to evaluating the current node. Used in bottom-up memory deallocation and mathematical syntax tree evaluation.

```
Tree Traversal Example:

       F
      / \
     B   G
    / \   \
   A   D   I

In-Order   (Left, Node, Right): A, B, D, F, G, I
Pre-Order  (Node, Left, Right): F, B, A, D, G, I
Post-Order (Left, Right, Node): A, D, B, I, G, F
```

3.4 Associative Mapping: Hash Tables

A **hash table** is an associative data structure designed to provide constant average-time $O(1)$ key search, insertion, and deletion. It maps keys to array index locations using a mathematical transformation called a hash function.

3.4.1 Hash Functions and Collisions

A hash function $h(k)$ maps an arbitrary key space k to a bounded integer range $[0, M-1]$, where M is the size of the underlying array table. A uniform hash function distributes keys evenly across all array indices to minimize collisions.

A **hash collision** occurs when two distinct keys evaluate to identical indices ($h(k_1) = h(k_2)$ for $k_1 \neq k_2$). Because key spaces are typically much larger than array size M, collisions are unavoidable due to the Pigeonhole Principle.

3.4.2 Collision Resolution Techniques

Separate Chaining

In separate chaining, each array slot acts as a reference to an independent linked list (or bucket) storing all key-value entries mapped to that index.

```
Separate Chaining Layout:

 Slot
+---+
| 0 | --> [ Null ]
+---+
| 1 | --> [ KeyA | ValA ] ---> [ KeyB | ValB ] ---> Null
+---+
| 2 | --> [ Null ]
+---+
```

Search time in a chained table depends on the average list length, governed by the **load factor** $\alpha = n/M$, where n is the total number of inserted keys and M is table capacity.

Open Addressing

Open addressing stores all entries directly inside the primary array. When a collision occurs, the algorithm systematically probes alternate slots until an empty cell is found:

- **Linear Probing**: Inspects sequential slots using index formula $h(k, i) = (h'(k) + i) \pmod{M}$ for probe step $i \in \{0, 1, \ldots, M-1\}$. This approach can lead to primary clustering, where contiguous blocks of occupied slots build up and slow down lookups.
- **Quadratic Probing**: Uses a non-linear quadratic offset $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod{M}$ to reduce primary clustering.
- **Double Hashing**: Uses a secondary independent hash function $h_2(k)$ to calculate probe step sizes: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \pmod{M}$.

3.4.3 Worked Example: Collision Resolution in Hash Tables

Insert integer keys $[15, 22, 8, 29]$ into a hash table of size $M = 7$ using hash function $h(k) = k \pmod 7$.

- Insert 15: $h(15) = 15 \pmod 7 = 1$. Slot 1 is empty. Place 15 at index 1.
- Insert 22: $h(22) = 22 \pmod 7 = 1$. Collision at slot 1.
- Insert 8: $h(8) = 8 \pmod 7 = 1$. Collision at slot 1.
- Insert 29: $h(29) = 29 \pmod 7 = 1$. Collision at slot 1.

Using **Linear Probing**:

1. Key 15 goes to slot 1.
2. Key 22 hashes to 1 (occupied). Probe index $(1 + 1) \pmod 7 = 2$. Place 22 at slot 2.
3. Key 8 hashes to 1 (occupied). Probe index 2 (occupied). Probe index $(1 + 2) \pmod 7 = 3$. Place 8 at slot 3.
4. Key 29 hashes to 1 (occupied). Probe index 2 (occupied), index 3 (occupied), index $(1 + 3) \pmod 7 = 4$. Place 29 at slot 4.

```
Linear Probing Result Table:
Index:   0      1      2      3      4      5      6
      +------+------+------+------+------+------+------+
Keys: | Null |  15  |  22  |  8   |  29  | Null | Null |
      +------+------+------+------+------+------+------+
```

Using **Separate Chaining**:

```
Separate Chaining Result Table:
Index 1: [ 15 ] ---> [ 22 ] ---> [ 8 ] ---> [ 29 ] ---> Null
```

3.5 Comparative Analysis and Systemic Applications

Selecting the right data structure requires evaluating operational performance, memory overhead, and implementation trade-offs.

*Note: Insertion and deletion operations for linked lists run in $O(1)$ time given a direct pointer reference to the target position; otherwise, locating the position requires $O(n)$ search time.

3.5.1 Integrated Software Applications

Real-world applications combine these core structures to manage operational tasks efficiently:

- **Text Editor Undo/Redo System**: Dual stack structures store operational state histories. Undoing an action pops state data from the undo stack and pushes it onto the redo stack.

Table 3.2: Algorithmic Time and Space Complexity Comparison

Data Structure	Average Access	Average Search	Average Insertion	Average Deletion
Singly Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^*$	$\Theta(1)^*$
Doubly Linked List	$\Theta(n)$	$\Theta(n)$	$\Theta(1)^*$	$\Theta(1)^*$
Stack (Array/List)	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Queue (Array/List)	$\Theta(n)$	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$
Binary Search Tree	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$	$\Theta(\log n)$
Hash Table	N/A	$\Theta(1)$	$\Theta(1)$	$\Theta(1)$

- **Task Scheduling and Breadth-First Search (BFS)**: Operating system process managers and graph traversal algorithms use queues to maintain execution order and process nodes sequentially.
- **Database Indexing and Symbol Tables**: Compilers use hash tables for constant-time lookup in scope symbol tables, while binary search trees support range queries and sorted index scans.

3.6 Conclusion

Concrete data structure implementations determine how efficient a program is in terms of speed and memory usage. Pointer-based structures manage dynamic memory allocation flexibly, restricted linear structures enforce strict access rules, hierarchical trees optimize ordered searching, and hash functions enable constant-time data retrieval. Understanding these internal mechanics provides the foundation for the next chapter, which covers choosing and tailoring the right data structure to solve complex, real-world problems.

Tutorial Questions

1. Compare the operational time complexity and memory layouts of dynamic arrays and singly linked lists. Identify scenarios where each structure is preferred over the other.
2. Trace the following operations on an initially empty stack: `push(12)`, `push(7)`, `pop()`, `push(19)`, `push(3)`, `pop()`, `peek()`. Draw the stack layout at each step and state the value returned by the final `peek()` operation.
3. Explain why a standard array implementation of a queue can result in an $O(n)$

dequeue runtime performance. Show how a circular array structure resolves this issue using modular arithmetic.

4. Evaluate the following postfix expression step-by-step using a stack, showing the stack state after reading each token:

$$8 \quad 4 \quad 2 \quad / \quad * \quad 3 \quad + \quad 5 \quad -$$

5. Given an unweighted binary search tree containing keys $[50, 25, 75, 10, 30, 60, 80]$, construct the resulting tree topology. Demonstrate step-by-step node updates when deleting key 25.
6. Insert keys $[18, 26, 35, 9, 44, 27]$ into a hash table of size $M = 7$ using hash function $h(k) = k \pmod 7$. Draw the resulting table structures under:
   (a) Linear Probing Open Addressing.
   (b) Separate Chaining.
7. A system developer uses a binary search tree to store symbol table keys. Under worst-case insertion conditions, operations degrade to $O(n)$ time. Explain why this degradation occurs and propose an architectural modification to ensure worst-case $O(\log n)$ performance.
8. High-performance caching layers use a Least Recently Used (LRU) policy, which requires fast element access and fast structural updates. Identify two data structures covered in this chapter that can be combined to support $O(1)$ search and $O(1)$ displacement operations, and explain how they interact.